

Developer Code of Conduct

Team Communication

Teams, Skype, Email, Jira, etc and notes on each of these and what we use it for.

Sprints

Notes around sprints and what meetings occur when. What window do we have for testing? What environments does each part of the sprint point to? When's the show-and-tell? Who's involved in each meeting.

Two week sprints:

- Sprints start on Tuesday, end on Monday.
- Sprint walkthrough ideally two weeks before the start of the sprint.
- Retrospective on first day of the new Sprint (Tuesdays)

Release Notes

Preparing release notes with completed Jira tickets before show-and-tells based on `development`. Each item corresponds to Jira tickets in the On Development state.

SENS-199 Task Title — any additional notes as required.

Jira

 Before starting the work ensure you have read [Jira Ticket Creation and Update](#)

Bug Priorities

- Highest: This level is reserved for critical issues that have severe consequences, such as data loss, downtime, or a massive impact on customers. These issues require immediate attention and should be addressed within the current sprint, preferably **within 48 hours**.

- **High:** High-priority issues that impact customers fall into this category. While they are not as critical as the highest level, they still need to be addressed promptly. The goal is to resolve these issues within the current sprint, typically **within one to three weeks**.
- **Medium:** Issues categorized as a medium priority can be scheduled for resolution in an upcoming sprint. They are important but not as time-sensitive as the highest or high-priority issues. These can typically be addressed **within six or more weeks**.
- **Low:** Issues labelled as low priority are not required to be fixed within a specific timeframe. They are of lesser importance and can be addressed as resources and time allow. These issues may be deprioritized in favour of higher-priority items.
- **Lowest:** This level is currently unused, indicating that there is no designated priority for such issues at the moment. It could imply that these issues are not actively considered or that they are of very low significance.

By organizing bugs into these priority levels, teams can allocate resources and address critical issues promptly while managing less urgent ones in a structured manner.

States

Jira tickets go through different states as they progress through their lifecycle. Each state has a specific meaning and corresponds to a certain git branch. Here is an enhanced description of each state.

- **Design** - Tickets that are being created, and not ready for development yet.
- **To Do** - Untouched tickets. This state represents tickets that are yet to be picked up for work, or tickets requiring rework. These tickets are waiting for someone to start working on them. Take a branch off master and start working on these in order of priority, and the order they are in the sprint.
- **In Progress** - When a ticket is in progress, it means that it is actively being worked on. These tickets are typically associated with a feature-specific git branch, named "sen-xxxx-feature-name," where "xxxx" represents a unique identifier for the ticket and "feature-name" describes the specific feature being developed much like `feature/sen-xxxx-feature-name`.
- **Dev Review** - Tickets in this state require developer review before they can progress further, and have a PR from `feature/sen-xxxx-feature-name` into `development`. A pull request (PR) is created from the feature branch ("sen-xxxx-feature-name") into the development branch. The ticket remains in this state until the PR is reviewed and approved by the developers. Tickets should ideally only stay in this state for an hour or two at most.

- **Dev QA** - Once the PR is approved, the ticket is merged into the development branch. Tickets are checked into the `development` branch and live in the **Development** environment. At this stage, the changes associated with the ticket are included in the development branch of the git repository. The ticket is considered to be "on development" and the changes are reflected in the Development environment. This is where the first QA pass happens.
- **On QA** - Tickets are checked into the `qa` branch and live in the **QA** environment. Meaning, tickets in this state have been checked into the QA branch, which is a dedicated branch for testing purposes. The changes associated with the ticket are deployed to the QA environment for thorough testing and verification
- **UAT** - Tickets that have passed the QA verification process are marked as "UAT" This indicates that the changes have been successfully tested and meet the desired quality standards.
- **Ready For Release** - Tickets in this state indicate that the work on the ticket has been completed. Once a ticket reaches this state, it is considered resolved.
- **Won't Fix** - This state is used for tickets that cannot be reproduced or are deemed not to be an issue that needs to be resolved. These tickets are closed without any further action. They are typically associated with the branch and environment in which they were being worked on before the decision was made not to fix them.

By following this flow, Jira tickets move through various states, allowing teams to track their progress and ensure a smooth development and QA process.

Types

Details on each issue type and how they fit into our development.

Bugs:

- Bugs are issues that are identified during the development process or reported by users after deployment.
- They represent errors or malfunctions in the software that require developer re-work to fix.
- Bugs do not count towards the velocity, which is a measure of the amount of work completed in a given time period.
- Examples of bugs could be a feature not working as intended, a crash or freeze in the software, or incorrect data being displayed.

Tasks:

- Tasks are issues that do not require any work from a developer's perspective.

- They serve as reminders or non-technical actions that need to be taken.
- Tasks can be used to coordinate with other teams or stakeholders, such as following up with the marketing department on branding guidelines.
- Unlike bugs or stories, tasks do not involve developer work and therefore do not count towards velocity.



- e.g. "Follow-up with marketing on branding guidelines."

Enhancements/Improvements:

- Stories are issues that require developer work and are estimated based on their complexity.
- They represent the main units of work in an agile development process and are typically user-focused.
- Stories are used to define and deliver specific features or functionality to the users.
- They are estimated in terms of the effort required and contribute to the velocity of the development team.
- Examples of stories could be implementing a new user registration process, adding a search feature, or improving the user interface.

Epics:

- Epics are groups of related stories that share a common goal or theme but may not be constrained to a single sprint.
- They represent larger pieces of work that span across multiple iterations or releases.
- Epics provide a way to organize and track progress on a higher-level objective.
- Each epic can be divided into multiple stories that contribute to achieving the overall goal.
- In the Confluence document, epics serve as the bigger picture or overarching chunks of work.

In the Confluence document:

- Epics are used to represent the larger objectives or themes of the project.
- Each epic can have multiple tickets associated with it, representing the detailed work required to achieve the epic's goal.
- Enhancements/Improvements provide the "meat" or substance of the Confluence document, outlining the specific features, functionality, or improvements being implemented.
- The Confluence document serves as a centralized repository of information, allowing stakeholders and team members to understand the overall project, track progress, and collaborate effectively.

Estimation

- "1 - half day": In this case, 1 point represents half a workday, which would be 4 hours (since 0.25 points = 1 hour). Therefore, a ticket with an estimation of 1 point would require 4 hours to complete.
- "2 - full day": Similarly, a ticket with an estimation of 2 points would represent a full workday, which would be 8 hours (2 points x 4 hours per point).
- "4 - 2 days": A ticket with an estimation of 4 points would require two full workdays to complete, totalling 16 hours (4 points x 4 hours per point).

 Keep in mind that these estimations are based on the assumption that 0.25 points correspond to 1 hour in your specific points system.

Transitions

Transitioning between issue states typically involves specific actions and information updates to ensure clear communication and effective tracking of progress. Here are the expected practices for transitioning between different issue states:

1. Resolving an Issue:

- Provide a detailed comment describing the cause of the issue and the steps taken to fix it. This comment should include any relevant information that helps others understand the resolution process.

2. Transitioning to In Progress:

- Assign the task to yourself if it is not already assigned. This helps in clearly indicating the person responsible for working on the task.

3. Transitioning from In Progress:

- Add Time Tracking information when transitioning away from the In Progress state. This involves logging the actual effort spent on the task, such as the time worked or the number of hours spent.

 These practices aim to maintain a comprehensive history of the issue's progress and facilitate better collaboration within the team. By providing clear and detailed comments, assigning tasks, and tracking time, it becomes easier to understand the status of the issues and ensure accountability in the resolution process.

Source Control

Notes around GitLab, Github. See <https://support.atlassian.com/jira-software-cloud/docs/reference-issues-in-your-development-work/> for details about referencing tickets in branches and commits.

- All commits need to include the **Jira ticket ID and time tracking**. i.e. `SENS-54 #time 45m Started work on Foo feature and added the landing page.` For info on time tracking in commits, see [Smart Commits](#). You'll need to add your Jira email address to GitHub for this to work: see [Adding Emails](#).
- All commits to the `main / master` branch that deploys to Production must be via a PR, and the code changes reviewed by a senior member of the development team.

Branching Structure

Main development branch for the current sprint: `development`. Branch for QA at the end of each sprint show-and-tell: `qa`, merged by senior team member from `development`. Branch for production: `main`, merged via a senior team member through a PR from `qa`.

For training and demos, a new `sandbox` environment will be spun up running from the `main` branch (Production code) with demo data.

Some older repositories will use `master` instead of `main`.

As a general workflow, you'll:

- Pull down `development` to get all up-to-date changes.
- Assign a Jira ticket to yourself and mark it as In Progress.
- Create a new local branch named `feature/sens-xxxx-feature-name` based on the Jira ticket. i.e. Jira ticket SEN-54 titled "Add feature Foo with Bar capability" would be the `feature/sens-54-foo-with-bar` branch.
- Commit and push work regularly, at minimum once a day. Don't leave work uncommitted at the end of your work day. Don't leave commits locally without pushing them.
- Each commit message must contain both the Jira ticket you're working on and a description of what you did. i.e. "SENS-54 #time 45m Started work on Foo feature and added the landing page."

- If your work relates to a subtask, include both the subtask and parent task in the commit message. i.e. “SENS-54 SENS-60 #time 6h Added Bar capability to the Foo feature.”.
- Each commit should include work on as fewer Jira tickets as possible. Do not commit monolithic commits that complete multiple Jira tickets. Instead, commit more regularly to each individual feature. You should never get into a situation where a commit has “SENS-54 SENS-1 SENS-40 SENS-50 Finished up everything.”.
- If your feature requires review, create a new PR from your feature branch into `development` and mark your Jira ticket as needing review. The PR should thoroughly explain what you’ve done, what to test and the steps for another engineer to test out and accept.
- Where a bug is fixed and the issue is either critical and/or the fix is not self-evident, add a comment to the Jira ticket explaining what went wrong and what steps you did to resolve this.

Coding Standards

- Coding standards are a set of guidelines and best practices that developers follow to ensure code quality, maintainability, and security. Based on the points you've mentioned, here's a summary of coding standards to consider:
 - Security:**
 - Write secure code by assuming that someone will try to exploit it. Validate and sanitize all inputs and implement necessary security measures.
 - Minimize trust in client-side inputs and perform server-side validation and verification.
 - Grant permissions at the minimum necessary level and ensure that access controls are properly implemented.
 - Compiler Usage:**
 - Utilize the full potential of the compiler to catch errors and issues at compile time. Enable strict compiler settings and take advantage of static typing and type checking.
 - Knowledge Sharing:**
 - Share newly discovered techniques or best practices with others to promote learning and collaboration within the team or community.
 - Conventions and Standards:**
 - Follow standard conventions and established best practices rather than reinventing solutions. Consistency in code style and structure enhances readability and maintainability.
 - Self-Explanatory Code:**
 - Strive to write code that is easy to understand without excessive comments. Use meaningful variable and function names, write clear and concise code, and favor

readability over cleverness.

f. **Learning from Mistakes:**

- Learn from your own mistakes and help others by sharing your experiences. If a mistake is simple, fix it, but for more complex issues, take the opportunity to teach and educate others.

g. **Simplicity:**

- Keep code simple and avoid unnecessary complexity. Simplicity improves code readability, reduces bugs, and eases debugging efforts. Your future self and other developers will appreciate clean and straightforward code.

h. **User-Provided Code:**

- Avoid executing arbitrary code from users as it poses a significant security risk. Validate and sanitize user inputs thoroughly to prevent code injection and other vulnerabilities.

i. **Abbreviations:**

- Minimize the use of non-standard abbreviations in code. Choose descriptive and meaningful names for variables, functions, and classes to enhance code comprehension.

j. **Program Flow:**

- Avoid double negatives and convoluted program flow that might confuse readers or introduce bugs. Use clear and straightforward logic to improve code readability and maintainability.

k. **Credentials Protection:**

- Never include or check in sensitive credentials (such as passwords, access keys, or tokens) to source control repositories. Use secure and proper methods for managing credentials, such as environment variables or secure configuration files.

l. **Deployment Timing:**

- Avoid deploying code on Mondays or Fridays, as these days are typically associated with higher risks and increased chances of disruption. Choose a more stable time frame for deployments to minimize potential issues.

 These coding standards aim to promote secure, maintainable, and readable code while fostering a collaborative and learning-oriented development environment.

Secure-by-design principles

Secure-by-design principles are a set of principles and practices that guide the development of secure systems from the ground up. These principles help ensure that security is considered at every stage of the design and development process, rather than being an afterthought. Let's go through each of these principles:

1. **Minimize attack surface area:** This principle emphasizes reducing the potential points of vulnerability in a system. By minimizing the attack surface area, you decrease the number of potential entry points that attackers can exploit, thereby reducing the overall risk.
2. **Establish secure defaults:** Secure defaults refer to the practice of configuring systems and applications with the most secure settings by default. This ensures that even if users or administrators are unaware of specific security configurations, they still start with a strong security posture.
3. **The principle of least privilege:** This principle states that individuals or processes should only have the minimum level of privileges necessary to perform their tasks. By granting only the necessary permissions and access rights, you limit the potential damage that can be caused if a user's account is compromised.
4. **The principle of defence in depth:** Defense in depth involves implementing multiple layers of security controls to provide overlapping protection. This approach assumes that no single security measure is foolproof, so by having multiple layers, if one layer is bypassed or compromised, other layers can still provide protection.
5. **Fail securely:** This principle focuses on handling errors and failures in a way that preserves system security. By implementing proper error handling mechanisms, such as graceful degradation and fail-safe defaults, you can prevent security vulnerabilities that may arise from unexpected failures.
6. **Don't trust services:** This principle promotes the idea of not blindly trusting external services or components. It emphasizes the need to verify and validate the security of third-party services, libraries, and dependencies before integrating them into a system.
7. **Separation of duties:** Separation of duties involves assigning different responsibilities to different individuals or roles. This ensures that no single person has complete control or knowledge over critical system functions, reducing the risk of abuse or unauthorized access.
8. **Avoid security by obscurity:** This principle advises against relying on secrecy or obscurity of system design or implementation as the primary means of security. Instead, security should be based on sound cryptographic principles, strong authentication mechanisms, and secure protocols.

9. **Keep security simple:** Simplicity is key to effective security. Complex systems are more prone to vulnerabilities and misconfigurations. By keeping security mechanisms simple, it becomes easier to understand, implement, and maintain them properly.
10. **Fix security issues correctly:** When security vulnerabilities or weaknesses are identified, they should be addressed promptly and correctly. This involves employing proper vulnerability management processes, conducting thorough security testing, and applying appropriate patches or fixes.

 By adhering to these secure-by-design principles, organizations can significantly enhance the security posture of their systems and reduce the likelihood of successful attacks.

Privacy-by-design principles

Privacy-by-design principles are a set of guidelines that promote privacy and data protection throughout the entire lifecycle of a system, product, or service. These principles ensure that privacy considerations are incorporated into the design and development processes from the outset, rather than being addressed as an afterthought. Let's explore each principle in more detail:

1. **Proactive not Reactive; Preventative not Remedial:**

This principle emphasizes the importance of anticipating and addressing privacy risks in advance, rather than reacting to privacy issues after they occur. By adopting a proactive and preventative approach, privacy is integrated into the system's design, mitigating potential privacy breaches before they happen.

2. **Privacy as the Default Setting:**

Privacy should be the default option, meaning that the maximum level of privacy protection is automatically applied to users' personal information. Users should not be required to take extra steps to protect their privacy, such as opting out of data collection or adjusting privacy settings. Instead, privacy should be the norm, and users can make informed decisions to share their data if they wish.

3. **Privacy Embedded into Design:**

This principle highlights the need to integrate privacy features and safeguards directly into the design of systems, products, or services. Privacy considerations should be a fundamental part of the design process, ensuring that privacy protections are inherent and not added as an afterthought or bolt-on feature.

4. **Full Functionality – Positive-Sum, not Zero-Sum:**

Privacy-by-design principles aim to strike a balance between privacy protection and the functionality and utility of a system. It encourages developers to find solutions that provide

comprehensive privacy while still offering the desired features and benefits to users. The goal is to achieve a positive-sum outcome where privacy and functionality can coexist.

5. End-to-End Security – Full Lifecycle Protection:

This principle emphasizes the need for security measures and privacy protections throughout the entire lifecycle of data. It includes secure data collection, storage, processing, sharing, and disposal. By addressing privacy and security at every stage, the risks of unauthorized access or misuse of personal information are minimized.

6. Visibility and Transparency – Keep it Open:

Transparency is crucial in privacy by design. Organizations should provide clear and easily understandable information about their data collection, use, and sharing practices. This includes transparent privacy policies, disclosure of data practices, and effective communication with users to ensure they have a clear understanding of how their data is handled.

7. Respect for User Privacy – Keep it User-Centric:

The final principle emphasizes the importance of respecting users' privacy preferences and maintaining user-centric design practices. User privacy choices and consent should be respected, and individuals should have control over their personal information. User-friendly interfaces and tools should be provided to enable users to manage their privacy settings effectively.

By following these privacy-by-design principles, organizations can build systems and services that prioritize privacy and data protection while still delivering valuable functionality and user experiences. These principles not only enhance privacy but also help to build trust with users and establish a privacy-conscious culture within an organization.

PR Review

To prepare the feature for a senior review, consider the following checklist:

1. Test coverage:

- Are there sufficient tests for all new code that has been written?
- Have the related tests been updated for any modified code?

2. Functionality:

- Does the feature work as described in the corresponding Jira ticket?
- Has the bug mentioned in the ticket been fixed?
- Have all parts of the Jira ticket been addressed?

- Does the new feature or bug fix work without introducing any additional issues?

3. **Code quality:**

- Is the code correct and functional?
- Are there any instances of dead code, such as unused imports, debugging functions, or commented-out code?
- Are the naming conventions and indentation consistent and appropriate?
- Have logging statements like `print`, `dump`, or `console.log` been removed from the committed code?
- If the sample code is left in commented lines, is it properly marked with a TODO comment and an explanation?

4. **Understanding:**

- Do you understand what the code is doing? If not, there may be a lack of understanding from the person who wrote it as well. Consider reaching out to the author for clarification.
- If the code's purpose is not clear, consider adding comments to explain its functionality.

5. **Design:**

- Does the approach taken to solve the problem make sense?
- Will this implementation require additional work in the future?
- Could the problem have been solved in a more efficient or elegant way?

 By addressing these points, you can ensure that the feature is well-prepared for the senior review and increase the chances of receiving valuable feedback and approval.